

Riassunti TWC

Davide Cuna

June 22, 2025

Contents

1 Syllabus: Contenuti Dettagliati	2
1.1 Cloud Computing	2
1.1.1 Concetti Generali del Cloud	2
1.1.2 Il Cloud nella Pubblica Amministrazione (PA)	2
1.1.3 Amazon Web Services (AWS) come Caso di Studio	2
1.1.4 Altre Piattaforme Cloud	3
1.1.5 Cloud, IoT e Big Data	3
1.2 Tecnologie Web Lato Server	3
1.2.1 Concetti Generali e Gestione dello Stato	3
1.2.2 PHP	4
1.2.3 Web Services	5
1.2.4 Frameworks Web	5
1.2.5 Altre Interfacce: Assistanti Vocali	6
2 Snippet	7
2.1 Cassaforte	7
2.2 Altro tipo di esercizio simile	7
2.3 Cookies	8
2.4 Dipendenti	9
2.5 Json	10
2.6 Funzioni	11
2.7 Logger	11
2.8 Matricola JSON	12
2.9 php JPEG	13
2.10 UpFile	15
2.11 Web Services	17
2.12 Shortcut per clients	17
3 Domande di teoria	18

1 Syllabus: Contenuti Dettagliati

Gli argomenti specifici trattati nel Syllabus sono suddivisi in due macro-aree principali: Cloud e Tecnologie web lato server.

1.1 Cloud Computing

Questa sezione introduce i concetti, i servizi e le applicazioni del cloud computing, con un focus sul caso di studio AWS.

1.1.1 Concetti Generali del Cloud

Il Cloud Computing implica l'astrazione e virtualizzazione di risorse informatiche come memoria, potenza di calcolo, capacità di rete e applicazioni. L'idea è di trattare l'ICT come un'utility, pagando per l'uso effettivo delle risorse (modello **pay-as-you-go**) e beneficiando di una disponibilità virtualmente illimitata e dell'elasticità. Si distingue tra **public cloud** (disponibile a pagamento) e **private cloud** (interna a un'organizzazione). Le ragioni per l'adozione del cloud includono il superamento dei limiti hardware, l'ottimizzazione dell'utilizzo dei server (che spesso sono sottoutilizzati) e la capacità di gestire picchi di carico senza investimenti ingenti. Il cloud sposta la computazione e la memorizzazione dal singolo computer a 'Internet'. Il concetto di cloud si interseca con altri paradigmi come Cluster Computing, Grid Computing, Utility Computing, Application Service Provider (ASP) e Content Delivery Network (CDN). Vengono analizzati i **modelli di servizio**:

- **SaaS (Software as a Service)**: utilizzo di programmi forniti e mantenuti in remoto (es. Google Docs, Office365).
- **PaaS (Platform as a Service)**: esecuzione remota di una piattaforma software per installare applicazioni (es. Heroku, Azure).
- **IaaS (Infrastructure as a Service)**: utilizzo di risorse remote di basso livello (es. server virtuali, spazio di memorizzazione, banda).

La scelta tra questi modelli dipende dal livello di controllo desiderato e dalla facilità di scalabilità. Gli attori principali sono il **Cloud Provider**, l'**Amministratore/Configuratore di soluzioni cloud** e l'**Utente finale**. Tra i **pregi** del cloud si annoverano l'aggiornamento centralizzato, la riduzione degli investimenti hardware, la manutenzione affidata a terzi e l'elasticità. I **problemi** includono la dipendenza da una connessione Internet continua, il rischio di **vendor lock-in** (difficoltà a migrare tra provider), la necessità di grandi aziende per la sostenibilità dell'infrastruttura e le complesse questioni di **privacy** e giurisdizione dei dati. Il Garante per la Privacy ha fornito indicazioni per un uso consapevole, sottolineando che l'utente finale mantiene comunque le responsabilità legali sulla protezione dei dati.

1.1.2 Il Cloud nella Pubblica Amministrazione (PA)

La **Strategia Cloud Italia** promuove la trasformazione digitale della PA, adottando il paradigma **cloud first** per lo sviluppo di nuovi servizi e l'acquisizione di software. Il censimento del Patrimonio ICT della PA ha rivelato carenze in sicurezza, affidabilità e scalabilità, con un scarso utilizzo del cloud. La strategia prevede la classificazione dei data center in Gruppo A (avanzati) e Gruppo B (da dismettere), con la migrazione prioritaria del Gruppo B al Cloud della PA. I vantaggi attesi per la PA, i cittadini e le PMI sono risparmi, maggiore efficienza, offerta di servizi digitali più ampia e sicura, e maggiore scalabilità. La strategia differenzia tra **Dato Strategico, Critico** e **Ordinario** in base all'impatto di una loro compromissione. Per i dati strategici e critici, è previsto il **Polo Strategico Nazionale (PSN)**, un'infrastruttura cloud privata o ibrida; per i dati ordinari, si useranno provider certificati.

1.1.3 Amazon Web Services (AWS) come Caso di Studio

AWS è presentato come un caso di studio concreto di servizio cloud. Nato dall'infrastruttura interna di Amazon, è stato aperto al pubblico nel 2006 con S3. I servizi AWS sono costruiti con un approccio a **loose coupling**, accessibili tramite **web services** (SOAP o REST) e librerie specifiche. I costi sono basati sul modello **pay-as-you-go**, dipendendo da tempo di CPU, volume di GB trasferiti, numero di chiamate, spazio di memorizzazione e altri fattori. L'architettura AWS è suddivisa in **Regioni** e **Availability Zones** interconnesse, permettendo agli sviluppatori di scegliere la localizzazione delle proprie applicazioni e dati per ragioni legali o di vicinanza al mercato. La sicurezza è gestita tramite Access Identifiers, Access Control Lists (ACL) e Identity and Access Management (IAM). I servizi AWS trattati includono:

- **EC2 (Elastic Compute Cloud)**: offre infrastruttura di calcolo sotto forma di **istanze server virtuali**. Le istanze sono create da **AMI (Amazon Machine Image)** che includono S.O., server e ambienti di sviluppo. Le tipologie di istanze includono **On-demand**, **Reserved** e **Spot**, con costi che variano in base all'istanza, al S.O. e al software preinstallato. Le risorse delle istanze sono temporanee, mentre i **volumi EBS (Elastic Block Store)** offrono memoria persistente. Vengono menzionati anche **Elastic Load Balancer (ELB)**, **Cloudwatch** per il monitoraggio e **Auto Scaling** per la gestione automatica delle istanze in base al carico. Si impara a creare un web server con Apache su EC2.

- **S3 (Simple Storage Service)**: un file system virtuale accessibile via Web per la memorizzazione di dati binari. Gli oggetti sono organizzati in **bucket**. Viene menzionato **S3 Glacier** per l'archiviazione a basso costo di dati cui si accede raramente. Vengono illustrate l'installazione e l'uso dell'SDK per PHP per listare bucket, oggetti e per effettuare upload. Vengono evidenziate le attenzioni sui costi, sul tempo del server e sulla latenza.
- **DynamoDB**: un **database NoSQL serverless** con scalabilità automatica e alto throughput. Supporta sia documenti che modelli chiave-valore. Vengono presentati esempi in PHP per la creazione di tabelle, l'inserimento ('putItem'), la ricerca ('query') e la scansione ('scan') di dati. Si distingue da RDS, che offre soluzioni relazionali.
- **Lambda (Serverless Computing)**: il servizio serverless di base di AWS, che implementa il concetto di **Function as a Service (FaaS)**. Il provider gestisce completamente il server, e il cliente paga solo per le risorse consumate dall'applicazione, non per il tempo di accensione della macchina. Il funzionamento è basato su **eventi** che fungono da trigger per l'attivazione del codice ("funzione lambda"). Un esercizio pratico prevede la creazione automatica di miniature di immagini caricate in S3 tramite una funzione Lambda in Python. Viene descritta la configurazione dei permessi tramite IAM (Policy e Ruoli), la struttura del codice della funzione ('lambda.handler', 'event', 'context') e l'uso di **Layers** per le librerie. Viene anche accennato all'orchestrazione di servizi con **AWS Step Functions**.
- **CloudFront**: una **Content Delivery Network (CDN)** che replica gli oggetti in server distribuiti geograficamente "vicini" agli utenti per migliorare l'accessibilità e le prestazioni.

1.1.4 Altre Piattaforme Cloud

Vengono brevemente menzionate altre importanti piattaforme cloud:

- **Microsoft Azure**: offerta cloud di Microsoft, inizialmente orientata a PaaS, ora anche IaaS, supportando Windows e Linux. Offre macchine virtuali, storage, database (SQL Server, MySQL, Postgres, NoSQL) e ha un meccanismo di free tier simile ad AWS.
- **Google Cloud**: offre crediti e servizi gratuiti entro certi limiti.
- **Heroku**: un servizio **PaaS** basato su Amazon EC2, per lo sviluppo, pubblicazione ed esecuzione di applicazioni in vari linguaggi e framework. Sfrutta **Git** per il deployment e introduce concetti come **slug** (immagine compressa dell'applicazione) e **dyno** (singoli processi). Permette l'integrazione con S3 per i contenuti statici e offre numerosi **add-ons** per funzionalità aggiuntive (database, monitoraggio, ecc.). Vengono discussi i pro e i contro di Heroku.

1.1.5 Cloud, IoT e Big Data

Il corso esplora anche la convergenza del cloud con l'Internet of Things (IoT) e i Big Data.

- **Cloud e IoT**: l'**Internet of Things** estende la rete a oggetti e luoghi fisici, permettendo loro di comunicare e interagire. Le applicazioni includono domotica, agricoltura, smart city. Le tecnologie essenziali sono miniaturizzazione, sensoristica e comunicazioni a basso consumo. I sistemi cloud come AWS Lambda e API Gateway possono reagire a eventi provenienti da dispositivi IoT. Servizi specifici come **AWS IoT Core** gestiscono la registrazione dei dispositivi, lo storage dei dati, l'analytics e la comunicazione sicura. Vengono citati casi di studio come The Things Network (LoRaWAN) e piattaforme di dashboard e analytics come Grafana e Node-Red. Viene presentato un esempio di progetto locale, PollicIoT, un sistema per l'assistenza a malati di Alzheimer basato su wearable, rete LoRaWAN e servizi cloud.
- **Cloud e Big Data**: i **Big Data** sono collezioni di dati così estese da richiedere metodi e tecnologie specifiche (calcolo parallelo, cloud, statistica inferenziale, AI). Il cloud è ideale per applicazioni computazionalmente costose ma non continue, come la fase di training del **deep learning**. Vengono menzionate le offerte di **GPU in cloud** (es. AWS EC2 con AMI specializzate, AWS Sagemaker) e piattaforme gratuite come Google Colaboratory. Lo storage per Big Data include database document-based (NoSQL) e **Data Lakes** (repository centralizzati per dati strutturati e non strutturati). Esistono piattaforme cloud dedicate all'elaborazione e analisi dei dati (es. AWS Kinesis, Azure Analytics Services) e sistemi di AI specifici (IBM Watson, AWS Rekognition).

1.2 Tecnologie Web Lato Server

Questa sezione esplora le tecniche e i linguaggi per la creazione dinamica di risorse web, la gestione dello stato e i diversi tipi di servizi web.

1.2.1 Concetti Generali e Gestione dello Stato

La **programmazione lato server** si riferisce all'insieme di tecniche che permettono di creare dinamicamente risorse (come HTML, CSS, immagini) da distribuire tramite un server Web. Tali programmi sono attivati dal server su richiesta. Le loro utilità includono l'accesso a basi di dati tramite interfaccia Web, la creazione di pagine dipendenti dal contesto (utente, data, browser) e lo sviluppo di applicazioni interattive. I **limiti** della programmazione solo lato server (senza

l'ausilio di client-side) includono un'interattività limitata (ogni input utente richiede una sostituzione dell'intera interfaccia) e la mancanza di sessioni, dove ogni pagina visitata è una "cosa diversa" senza memoria dello stato precedente. L'**input dell'utente** viene fornito al server tramite form HTML, utilizzando i metodi **GET** (parametri nell'URI) o **POST** (parametri nel corpo della richiesta HTTP). Vengono discusse le differenze e le raccomandazioni d'uso (es. GET non per produrre effetti sul server, POST per dati lunghi o sensibili). I **cookie**, introdotti da Netscape, sono piccole informazioni che il server può chiedere al client di memorizzare per ritrovarsi in visite successive. Sono variabili residenti sul client, utili per mantenere memoria tra visite o per tracciare il comportamento degli utenti. Un cookie è caratterizzato da dominio, path, data di scadenza, nome e contenuto. Le **sessioni** permettono di mantenere uno stato tra diverse visite dello stesso client, in modo più sicuro e completo rispetto ai soli cookie. Una sessione corrisponde a un insieme di variabili residenti sul server, associate a un identificatore univoco (SessionId) inviato al client. Le sessioni sono utili per autenticazione, pagine a contenuto differenziato per utenti, e form multipagina. In PHP, le sessioni si gestiscono con 'session_start()' e l'array superglobale '\$_SESSION'. Viene presentato un esempio di sistema di autenticazione basato su sessioni e utenti memorizzati in file di testo, evidenziando problemi di sicurezza e soluzioni. Viene anche mostrato come impedire il caching di pagine generate dinamicamente e un esempio di carrello in un e-shop implementato con le sessioni.

Il **Common Gateway Interface (CGI)** è stato storicamente il primo metodo per generare dinamicamente risorse Web. Un programma CGI viene lanciato dal server a seguito di ogni richiesta del client, il che lo rende computazionalmente costoso e lento. Il programma CGI è identificato da un URI ma non viene restituito; piuttosto, viene eseguito, e il suo output costituisce la risposta per il client. Per motivi di sicurezza, i programmi CGI risiedono in una directory protetta ('/var/www/cgi-bin/'). I dati in input vengono passati tramite variabili d'ambiente o standard input. Qualsiasi linguaggio di programmazione può essere usato per i programmi CGI, con Perl che è stato storicamente molto diffuso. Il **server Web Apache** è un demone che gestisce il protocollo HTTP. Le sue attività includono la traduzione di URI, la verifica dell'identità dell'utente e dei privilegi di accesso, la determinazione del tipo MIME e la registrazione degli eventi in file di log. Apache è modulare, con ogni passo gestito da uno o più moduli. Vengono illustrate le directory e i file di configurazione importanti ('/var/www/', '/etc/apache2/', '/var/log/apache2/'). Si discute l'esecuzione di Apache, con un processo padre e processi figli che gestiscono le richieste, e l'importanza dei **privilegi dei file** per l'accesso alle risorse.

1.2.2 PHP

PHP, originariamente "Personal Home Page" e ora "PHP: Hypertext Preprocessor", è un linguaggio di **scripting lato server**. Gli script PHP sono mescolati al codice HTML e vengono interpretati da un modulo esterno del server HTTP. Il modulo sostituisce lo pseudo-tag PHP con l'output generato dallo script, mantenendo lo stato tra script multipli nello stesso documento. PHP è un linguaggio interpretato con sintassi di derivazione C, supportando sia lo stile procedurale che quello object-oriented. Dispone di una vasta gamma di librerie per gestione stringhe, accesso a database, XML, cookie e creazione dinamica di immagini. è open source e multiplatforma. Vengono illustrate le **sintassi di base** per commenti, tipi di dati (scalari e composti), variabili (con prefisso '\$'), costanti ('define'), e strutture di controllo ('if', 'switch', 'for', 'while'). Si discute l'uso delle **variabili predefinite** ('\$SERVER', '\$GET', '\$POST') per accedere ai parametri utente e delle **funzioni predefinite** come 'echo', 'print', 'phpinfo', 'isset', 'die'. La gestione dei form è un tema centrale, con la creazione di pagine bimodali che contengono sia il form HTML che la logica di elaborazione PHP. Vengono trattate la dichiarazione e l'uso delle **funzioni** (con concetti di scope locale e globale) e l'**inclusione di file** ('include()', 'require()', '_once') per la modularizzazione del codice. Un aspetto cruciale è l'emissione delle **intestazioni HTTP** tramite la funzione 'header()', che deve avvenire prima di qualsiasi altro output. La gestione dei **cookie** lato server avviene tramite 'setcookie()' e l'accesso ai cookie tramite '\$_COOKIE'.

Vengono sviluppati **due casi di studio** estesi in PHP:

- **Galleria di Immagini**: un'applicazione web dinamica per gestire e navigare tra immagini, con generazione dinamica di link, accesso al filesystem del server e gestione dell'upload di file. Si discute la creazione di miniature "on-the-fly" o "reali" utilizzando la libreria GD di PHP. Vengono inoltre affrontati importanti aspetti di sicurezza legati all'upload di file e all'inclusione di file sensibili.
- **E-Shop (Negozio Online)**: un'applicazione che simula una vetrina prodotti, l'inserimento di prodotti e la navigazione nell'archivio. La prima versione usa il filesystem per memorizzare i prodotti, con un foglio di stile dinamico ('stile.php'). Vengono dettagliate le funzioni per la registrazione, lettura e conteggio dei prodotti, e la validazione dell'utente, con note sull'inefficienza di alcune implementazioni e sulla pulizia dei dati. L'introduzione del **carrello** nella versione 2 è un'applicazione pratica delle **sessioni** PHP.

Viene trattata la **configurazione degli errori** in PHP tramite 'php.ini', 'ini_set()' e 'error_reporting()', e le **funzioni utili per il debugging** come 'print_r()', 'var_dump()' e 'var_export()'. La **Programmazione Orientata agli Oggetti (OO)** in PHP viene introdotta con concetti di classi, proprietà, metodi, costruttori, distruttori e visibilità (public, protected, private). I **Namespaces** sono presentati come un meccanismo per risolvere le collisioni tra nomi di funzioni, classi e costanti, specialmente quando si usano librerie esterne. Viene mostrata la sintassi per la definizione e l'utilizzo dei namespaces. La gestione delle **librerie esterne** è un aspetto cruciale. Si discute **PEAR (PHP Extension and Application Repository)** come primo tentativo di sistema di installazione di codice PHP, ma l'attenzione si sposta su **Composer**, il **Dependency Manager for PHP**. Composer installa le librerie necessarie in una directory specifica del progetto ('vendor/') basandosi

sulle dipendenze dichiarate in un file 'composer.json'. 'composer.lock' garantisce la riproducibilità delle versioni esatte delle librerie, e 'vendor/autoload.php' include automaticamente tutto il necessario.

1.2.3 Web Services

Il corso dedica ampio spazio ai Web Services, esplorando l'architettura Service Oriented Architecture (SOA) e i due approcci principali: SOAP e REST.

SOA (Service Oriented Architecture) è uno stile architetturale che supporta la **service orientation**, favorendo la pubblicazione, descrizione, reperimento e utilizzo di servizi su una rete. Un **servizio** è un'entità autonoma, indipendente dalla piattaforma, in grado di svolgere funzioni di varia complessità. La **Service Orientation** è un paradigma di progettazione basato sulla composizione di servizi indipendenti. Vengono enunciati i **Principi della Service Orientation**: Standardized Service Contract, Loose Coupling, Abstraction, Reusability, Autonomy, Statelessness, Discoverability e Composability. Si discute il **Service Oriented Computing (SOC)** come disciplina più ampia che include SOA. Gli **obiettivi e benefici** del SOC includono maggiore interoperabilità, federazione, diversificazione dei fornitori (riduzione del vendor lock-in), allineamento tra business e tecnologia, aumento del ROI e agilità organizzativa. I **ruoli principali** in una SOA sono il **Service Provider**, il **Service Client** (o Requestor) e il **Service Registry**. Le **operazioni** chiave sono **publish** (descrizione e registrazione del servizio), **find** (reperimento e selezione del servizio) e **bind** (invocazione del servizio). Viene presentato un esempio di scenario aziendale per illustrare l'applicazione della SOA. Si analizzano i **livelli di adozione della SOA** in un'organizzazione. La **Quality of Service (QoS)** è la capacità di un servizio di rispondere alle aspettative del client e include dimensioni come disponibilità, accessibilità, conformità, performance, affidabilità, scalabilità, sicurezza e transazionalità. Il **Service-Level Agreement (SLA)** è un accordo formale sul livello di servizio fornito, includendo descrittori di qualità e penalità.

Si esaminano due approcci concreti ai Web Services:

- **Web Services "Standard" (basati su SOAP e WSDL)**: Definiti dal W3C come applicazioni software identificate da URI, con interfacce e connessioni descrivibili tramite XML e che supportano l'interazione diretta via messaggi XML su protocolli Internet. Le tecnologie di base includono HTTP, XML, **SOAP**, **WSDL** e **UDDI**. **SOAP (Simple Object Access Protocol)** è un protocollo per lo scambio di informazioni in ambiente distribuito, nato in Microsoft e IBM. Viene utilizzato prevalentemente per comunicazioni di tipo RPC (Remote Procedure Call) ma supporta diversi scenari. La struttura di un messaggio SOAP include un 'envelope' (busta) contenente un 'Header' (opzionale) e un 'Body' (con la procedura e i parametri). Vengono mostrati esempi di richieste e risposte SOAP. SOAP supporta anche **Attachments** per il trasferimento di dati binari. Gli errori sono gestiti tramite un messaggio 'Fault'. Vengono discussi i modelli di comunicazione **RPC-style** e **Document-style**. I **vantaggi** di SOAP includono semplicità (da XML), portabilità, compatibilità con firewall, standard aperto, interoperabilità e riconoscimento universale. Gli **svantaggi** riguardano la lentezza di HTTP come strato di trasporto, l'assenza di stato e la serializzazione per valore. **WSDL (Web Services Description Language)** è un linguaggio XML per la descrizione dei web services, specificando procedure, parametri e protocolli. Un documento WSDL è una definizione formale dell'API del servizio, con sezioni per Types, Messages, Port Types, Bindings e Services. Vengono citati i tool che aiutano nella generazione di stub e skeleton da WSDL. In PHP, il supporto standard per SOAP è disponibile tramite la libreria 'php-soap'.
- **Web Services REST (Representational State Transfer)**: REST è un paradigma generico dove lo stato dell'applicazione e le funzionalità sono rappresentati da **risorse accessibili tramite URI univoci**. I **RESTful Web Services** mappano le funzionalità su URI diverse e sfruttano la **semantica dei metodi HTTP** (POST per creare, GET per reperire, PUT per modificare, DELETE per rimuovere). Il risultato dell'invocazione di un servizio REST è tipicamente **XML o JSON**. **JSON (Javascript Object Notation)** è un formato standard di scambio dati basato su un subset di Javascript, che rappresenta oggetti (coppie nome/valore) e array (elenchi ordinati di valori). PHP fornisce funzioni ('json_encode()', 'json_decode()') per lavorare con JSON. La procedura per creare servizi REST implica l'identificazione delle risorse, la loro mappatura su URI significative e l'interpretazione del risultato da parte del client. Nel confronto **REST vs. SOAP**, JSON è più leggero ("fat free XML"), REST sfrutta nativamente la semantica HTTP, e la sicurezza si basa su quella di HTTP. SOAP è più standardizzato, con un ecosistema completo per la discovery e la gestione degli attachments, mentre REST manca di un contratto formale standardizzato (richiede documentazione scritta). A livello pratico, REST si basa su **URI significative**, l'uso completo dei **metodi HTTP** (GET, POST, PUT, DELETE) e delle relative **risposte HTTP** (es. 201 Created). Vengono affrontate le **problematiche di implementazione in PHP**, come la gestione dei metodi PUT e DELETE (che richiedono un parsing manuale del corpo della richiesta), e l'uso di **URL significative** tramite il modulo **mod_rewrite** di Apache e file **.htaccess**. Viene mostrato come ricostruire uno script PHP ('service.php') per gestire risorse RESTful in base al metodo HTTP e come impostare il 'Content-Type' della risposta (XML o JSON). Per il client, si accenna all'uso di librerie esterne o framework REST come SLIM. La documentazione dei servizi REST può essere facilitata da strumenti come **OpenAPI / Swagger**.

1.2.4 Frameworks Web

I **frameworks web** sono insiemi di strumenti e librerie che semplificano la creazione di applicazioni web, automatizzando operazioni ripetitive come la gestione delle richieste HTTP, la connessione ai database e la gestione di form e sessioni. Offrono vantaggi in termini di rapidità di sviluppo, sicurezza integrata, riutilizzo del codice e applicazione di standard.

Vengono discussi specifici framework:

- **Laravel (PHP)**: un framework PHP basato su Symfony, che segue l'architettura **MVC (Model-View-Controller)**. Il **Model** gestisce dati e logica di business, la **View** presenta i dati all'utente (interfaccia grafica) e il **Controller** riceve le richieste, interagisce con il Model e restituisce i risultati alla View. Il **Routing** in Laravel é il punto di ingresso che instrada le richieste al Controller appropriato. Laravel include un ORM integrato chiamato **Eloquent** per la gestione dei database relazionali e una versione semplificata (**Lumen**) per microservizi e API. Tra i suoi **vantaggi** figurano l'alta scalabilità, lo sviluppo rapido, la facile integrazione con altre librerie e framework JS (es. React, Vue), la gestione semplificata di autenticazione e autorizzazioni tramite middleware, le migrazioni per lo schema del database, il Query Builder, la validazione, le notifiche multicanale, la gestione file (locale o cloud), il Task Scheduling e il Queue system. I prerequisiti per Laravel includono PHP e Composer, oltre a specifiche estensioni PHP. La gestione del progetto avviene tramite **Composer** e l'interfaccia a linea di comando **Artisan CLI**. Vengono illustrate le configurazioni per il deployment (es. su Apache) e l'uso di Database Seeder per popolare il database.
- **Flask (Python)**: un **micro-framework** per lo sviluppo di applicazioni web in Python. Il suo core include solo le funzionalità essenziali (routing, gestione richieste, template engine Jinja2), rendendolo minimale e flessibile. Il resto delle funzionalità si aggiunge tramite un ricco ecosistema di **estensioni** (es. Flask-SQLAlchemy per i database, Flask-Login per l'autenticazione, Flask-WTF per i form). Flask é ideale per sviluppare applicazioni web dinamiche e API, ed é particolarmente adatto per API REST, middleware e interfacce per modelli di machine learning. L'**architettura di Flask** si basa su un Application Object, Routing, Request e Response, e il Template Engine Jinja2. Viene dettagliato il **flusso di lavoro** di Flask, dalla ricezione della richiesta all'invio della risposta. Si insegna a installare e avviare Flask localmente, che include un **web server integrato** che gira su 'http://localhost:5000'. Vengono spiegate le **pagine web dinamiche** generate da Flask utilizzando **template HTML** (con il motore Jinja2 e i suoi delimitatori '% ... %' per la logica e ' ... ' per le variabili). Viene discussa l'**estensione dei template** tramite un layout base e blocchi ereditabili. La **gestione dei form** in Flask si avvale dell'oggetto 'request' per accedere ai dati inviati con metodi GET e POST. Vengono forniti esempi pratici di form, calcolatrice, scelta multipla e validazione dei dati. Infine, viene mostrato come integrare un **database SQLite** in un'applicazione Flask utilizzando Flask-SQLAlchemy, con esempi di aggiunta, modifica e eliminazione di utenti tramite API REST (POST, PUT, DELETE) e la loro visualizzazione in una pagina HTML.
- **Docker**: una **piattaforma open-source per la creazione di container** portabili, leggeri e autosufficienti. Si contrappone alla **virtualizzazione tradizionale** (VM) evidenziando che i container sono più leggeri, condividono il kernel del sistema operativo host e non richiedono un hypervisor completo, il che porta a migliori performance e un uso più efficiente delle risorse. I **vantaggi** di Docker includono la capacità di replicare l'ambiente di produzione, la modularità dei servizi, la rapidità di avvio dei container, l'efficienza nell'isolamento dei servizi, la portabilità e l'alta personalizzazione. L'**architettura di Docker** ruota attorno a tre componenti principali: **Docker Client** (interfaccia utente), **Docker Daemon** (il server che gestisce gli oggetti Docker e il ciclo di vita dei container) e **Docker Registry** (un repository di immagini Docker, come Docker Hub). Viene menzionato **Docker Desktop** per semplificare la configurazione su vari sistemi operativi. I **concetti di base** di Docker sono:
 - **Container**: pacchetti standardizzati, leggeri ed eseguibili che contengono software, dipendenze e configurazioni, isolando l'applicazione [328, 334]. Un container é un'immagine in esecuzione.
 - **Immagini**: file immutabili di sola lettura che servono come modello per creare i container. Sono costruite su strati, con ogni strato che rappresenta modifiche al filesystem. Le immagini possono essere create tramite **Dockerfile**.
 - **Volumi**: unità di storage persistente gestite da Docker, usate per salvare dati al di fuori del container, garantendo persistenza anche dopo la rimozione del container.

Vengono illustrate le **procedure di installazione** di Docker su Windows, macOS e Linux. Vengono presentati i **comandi principali** di Docker per la gestione di immagini e container (es. 'docker pull', 'docker run', 'docker images', 'docker container ls', 'docker logs', 'docker exec', 'docker rm'). Il **Dockerfile** é un file di testo contenente istruzioni per costruire un'immagine Docker personalizzata (es. 'FROM', 'WORKDIR', 'COPY', 'CMD', 'docker build'). La **gestione delle variabili d'ambiente** nei container Docker é cruciale per separare la configurazione dal codice e adattare l'app a diversi ambienti; può essere fatta tramite argomenti CLI, file '.env' o Docker Compose. **Docker Compose** é uno strumento per definire e gestire **applicazioni multi-container** tramite un unico file di configurazione YAML ('docker-compose.yml'). Permette di avviare, fermare e gestire tutti i servizi con un singolo comando. La configurazione di 'docker-compose.yml' include la definizione dei servizi, delle reti e dei volumi. Un esempio pratico illustra come creare un ambiente Docker completo con un web server PHP/Apache e un database MySQL, definendo un Dockerfile personalizzato e usando 'docker-compose.yml' per orchestrare i servizi e la loro comunicazione.

1.2.5 Altre Interfacce: Assistenti Vocali

Il corso tocca anche il tema delle interfacce vocali, che stanno diventando sempre più diffuse su smartphone e nuovi dispositivi dedicati (es. Google Assistant, Alexa, Cortana, Siri). Questi sistemi mirano a semplificare l'accesso a servizi

online. é possibile sia utilizzare le loro funzioni nelle proprie applicazioni che aggiungere nuove funzionalità accessibili dai dispositivi fisici tramite **Alexa Skills** e **Google Actions**. Un'applicazione per assistenti vocali é essenzialmente un'applicazione web che riceve parole e frasi riconosciute da un motore AI. I concetti chiave includono la **pubblicazione** della skill/action, l'**invocazione** (ció che la fa partire) e l'**intento** (ció che l'utente vuole ottenere, con possibili **slots** come parametri). Vengono confrontate le **Skills di Alexa** e le **Actions di Google** per diffusione e modalit  di attivazione. Un esercizio pratico guida alla creazione di una **Alexa skill** tramite la Developer Console, con la configurazione di intenti, handler e la gestione degli slot. Viene anche affrontato il tema del debugging per lo sviluppo di queste interfacce.

2 Snippet

2.1 Cassaforte

1. lo script riceve in input in modalit  POST un file caricato tramite un form HTML (non   necessario descrivere il form), assieme ad un codice utente nel campo CODICE
2. se il file non   presente oppure risulta vuoto, l'esecuzione termina e mostra una segnalazione di errore
3. altrimenti lo script stabilisce una connessione con S3 su AWS. Per semplicit , il config.php   da considerarsi gi  impostato, e con le sue chiavi sar  possibile accedere in lettura e scrittura ad un bucket denominato CAVEAU
4. Lo script caricher  nel bucket il file ricevuto, dandogli come nome la concatenazione del codice utente e del timestamp corrente, separati da un underscore
5. Al termine del caricamento si deve far comparire un messaggio che ricorda all'utente il nome del file originale e quello assegnato al file per l'inserimento su S3

```
1 <html>
2 <head>
3 <title> Cassaforte inserisci </title>
4 </head>
5
6 <body>
7 <?php
8 require 'config.php';
9 require 'vendor/autoload.php'; //AWS SDK
10
11 use Aws\S3\S3Client;
12 use Aws\Exception\AwsException;
13
14 if($_SERVER['REQUEST_METHOD'] === 'POST') {
15     if(!isset($_FILES['file']) || filesize($_FILES['file']) === 0)
16         die("Errore:  File non ricevuto n");
17
18     $credentials = new Aws\Credentials\Credentials('chiave', 'segreto');
19
20     $s3 = new Aws\S3\S3Client([
21         'version' => 'latest',
22         'region' => 'eu-south-1',
23         'credentials' => $credentials
24     ]);
25
26     $file = $_FILES['file'];
27     $originalFileName = $file['name'];
28     $bucket = 'CAVEAU';
29     $codice = $_POST['CODICE'];
30     $time = time();
31     $newName = $codice . "_" . $time;
32
33     try {
34         $result = $s3->putObject(array (
35             'Bucket' => $bucket,
36             'Key' => $newName,
37             'SourceFile' => $file['tmp_name']
38         ));
39         echo "Il file originale '$originalFileName'   stato caricato come '$newName'.";
40     } catch (AwsException $e) {
41         die("Errore caricamento file AWS: " . $e->getMessage());
42     }
43 }
44 ?>
45 </body>
46
```

2.2 Altro tipo di esercizio simile

1. lo script riceve in input in modalit  POST un file caricato tramite un form HTML (non   necessario descrivere il form), assieme ad un codice utente nel campo CODICE;
2. se il file non   presente oppure risulta vuoto, o il codice non   presente, l'esecuzione termina e mostra una segnalazione di errore;

- altrimenti lo script stabilisce una connessione con S3 su AWS. Per semplicità, il config.php è da considerarsi già impostato, e con le sue chiavi sarà possibile accedere in lettura e scrittura ad un bucket denominato VAULT;
- lo script caricherà nel bucket il file ricevuto, dandogli come nome la concatenazione del codice utente e del nome file corrente, separati da un carattere "-";
- al termine del caricamento si deve far comparire un messaggio che ricorda all'utente il nome del file originale e quello assegnato al file per l'inserimento su S3, quest'ultimo linkato all'URL del file

```

1 <?php
2
3
4 require 'vendor/autoload.php';
5 include('config.php');
6 use Aws\S3\S3Client;
7 use Aws\Exception\AwsException;
8 if (!isset($_FILES['file']) || filesize($filename) == 0) {
9     die("Errore: file non presente o vuoto.");
10 }
11 if (!isset($_POST['CODICE'])) {
12     die("Errore: codice utente non presente.");
13 }
14 $codiceUtente = $_POST['CODICE'];
15 $file = $_FILES['file'];
16 $originalFileName = $file['name'];
17 $tempFilePath = $file['tmp_name'];
18 $bucket = 'VAULT';
19 $newFileName = $codiceUtente.'-'. $originalFileName; // Nome file da
20 caricare
21 try {
22     $credentials = new Aws\Credentials\Credentials('chiave',
23     'segreto');
24     $s3 = new Aws\S3\S3Client([ // Crea client S3
25         'version' => 'latest',
26         'region' => 'eu-south-1',
27         'credentials' => $credentials
28     ]);
29     $result = $s3->putObject([ // Carica il file nel bucket
30         'Bucket' => $bucket,
31         'Key' => $newFileName,
32         'SourceFile' => $tempFilePath,
33         'ACL' => 'public-read', // Opzionale: rendi il file
34 pubblico
35     ]);
36     $fileUrl = $result['ObjectURL']; // URL del file caricato
37     echo "Il file originale : $originalFileName<br>";
38     echo "Il file stato caricato come: <a
39 href=\"$fileUrl\">$newFileName</a>";
40 } catch (AwsException $e) {
41     echo "Errore durante il caricamento del file: " . $e -> getMessage();
42 }
43 ?>

```

2.3 Cookies

```

1 <?php
2 // Invio di un Cookie (nome Frollino) val cioccolato durata = 13 minuti
3
4 setcookie("frollino", "cioccolato", time() + (13 * 60), "/");
5
6 if(isset($_COOKIE[$cookie_name])) { // Verifica l'avvenuta impostazione del cookie
7     echo "Il cookie '$cookie_name' stato impostato con valore: " . $_COOKIE[$cookie_name];
8 } else {
9     echo "Il cookie '$cookie_name' stato impostato con valore: '$cookie_value' e durata: 13 minuti.";
10 }
11 ?>
12
13 <?php
14 // Lettura del cookie impostato precedentemente e salvataggio in variabile $colazione
15
16 $cookie_name = "frollino";
17 if (isset($_COOKIE[$cookie_name])) {
18     $colazione = $_COOKIE[$cookie_name];
19     echo "Il valore del cookie '$cookie_name' : " . $colazione;
20 } else {
21     echo "Il cookie '$cookie_name' non è impostato";
22 }
23 ?>
24
25 <?php
26 // Scrivere il codice PHP necessario per impostare ed inviare al client un cookie avente nome virus , valore pericoloso e dura
27 // Imposta il cookie
28 setcookie("virus", "pericoloso", time() + (17 * 60), "/");
29 // Verifica se il cookie stato impostato
30 if(isset($_COOKIE["virus"])) {
31     echo "Cookie impostato: " . $_COOKIE["virus"];
32 } else {
33     echo "Cookie non impostato.";
34 }

```

```

35 ?>
36
37 <?php
38 // Controllo se nella sessione esiste una variabile user con il valore pari ad hannibal. Se non esiste reindirizzare il browser v
39
40 session_start();
41 if (isset($_SESSION['user']) || $_SESSION['user'] !== 'hannibal') {
42     header('Location:login.php');
43     exit(); // Termina lo script
44 }
45 echo "Benvenuto, Hannibal!";
46 ?>
47
48
49 <?php
50 // PHP necessario per leggere il cookie impostato al punto precedente e salvarne il valore in una
51 // variabile di nome $home .
52
53 if (isset($_COOKIE['pagina_home'])) {
54     $home = $_COOKIE['pagina_home'];
55 } else {
56     $home = null; // Imposta $home a null se il cookie non esiste
57 }?>
58
59 <?php
60 // PHP necessario per memorizzare nella sessione una variabile counter con il valore pari a 100.
61 session_start();
62 // Memorizza nella sessione la variabile "counter" con valore 100
63 $_SESSION['counter'] = 100;
64 ?>
65
66
67 <?php
68
69 // 1. controllare se nella sessione esiste una variabile userlevel con valore maggiore di 3. Se non
70 // esiste, reindirizzare il browser verso una pagina login.php ; se esiste ma il valore inferiore a 3,
71 // emettere un segnale di errore;
72
73 session_start();
74 if (!isset($_SESSION['userlevel'])) {
75     header("Location:login.php");
76     exit();
77 } elseif ($_SESSION['userlevel'] < 3) {
78     die("Errore:livelloutenteinsufficiente.");
79 }
80 ?>
81
82 <?php
83
84 // 1. Scrivere il codice PHP necessario per controllare se nella sessione esiste una variabile
85 // SUPERUSER con valore maggiore di 3. Se esiste ma il valore inferiore a 3, reindirige il browser
86 // verso una pagina utentenumale.php; se non esiste, emettere un segnale di errore.
87
88 session_start();
89 if (!isset($_SESSION['SUPERUSER'])) {
90     die('Errore:La variabile di sessione SUPERUSER non esiste. ');
91 } elseif ($_SESSION['SUPERUSER'] <= 3) {
92     header("Location:utentenumale.php");
93     exit();
94 } else {
95     echo "Accesso consentito.";
96 }
97 ?>
98
99 <?php
100 session_start();
101 // variabile $_SESSION come $_FILES
102 if (!isset($_SESSION['privilegi'])) {
103     header('Location:login.php');
104     exit();
105 }
106 if($_SESSION['privilegi'] <= 5) {
107     die('Error:privilegi non sufficienti. ');
108 }
109 ?>

```

2.4 Dipendenti

1. lo script riceve in input in modalità GET il parametro cf che identifica un dipendente;
2. se il parametro cf non é presente oppure risulta vuoto, l'esecuzione termina e mostra una segnalazione di errore,
3. altrimenti lo script stabilisce una connessione con S3 su AWS (per semplicità, il config.php é da considerarsi già impostato);
4. Per ogni dipendente esiste un bucket il cui nome é dato dalla concatenazione di " stipendi- " e il parametro cf . Per semplicità, il bucket risulta accessibile in lettura con le chiavi già presenti nel config.php ;
5. lo script legge il contenuto del bucket, e mostra (in HTML) la lista dei file in esso contenuti, ma del solo tipo PDF (quindi con estensione .pdf).

- Ognuno dei file nella lista é "cliccabile", ed il link permette di scaricare il file stesso.
- Al termine dell'importazione si deve far comparire un messaggio che dice quanti stipendi sono presenti in tutto nel repository del dipendente.

```

1 <html>
2 <head><title>Dipendenti inserisci</title></head>
3 <body>
4
5 <?php
6 require 'vendor/autoload.php';
7 require_once("config.php");
8 use Aws\S3\S3Client;
9 use Aws\Exception\AwsException;
10
11 if (!isset($_GET['cf']) || empty($_GET['cf']))
12     die("Errore: Parametro cf non presente\n");
13 $cf = $_GET['cf'];
14
15 $credentials = new Aws\Credentials\Credentials('chiave',
16     'segreto');
17 $s3 = new Aws\S3\S3Client([
18     'version' => 'latest',
19     'region' => 'eu-south-1',
20     'credentials' => $credentials
21 ]);
22 $bucketName = "stipendi-" . $cf;
23 $objectsList = $s3->listObjectsV2(['Bucket' => $bucketName]);
24 $pdfFiles = [];
25 if (isset($objectsList['Contents'])) {
26     foreach ($objectsList['Contents'] as $object){
27         //if (substr($object['Key'], -4) === '.pdf')
28         if (mime_content_type($object) === 'application/pdf'){
29             $pdfFiles[] = $object['Key'];
30         }
31     }
32 }
33 echo "<h1>Lista dei file PDF per il dipendente con CF:
34     " . $cf . "</h1>";
35 echo "<ul>";
36 foreach ($pdfFiles as $file) {
37     $fileUrl = $s3->getObjectUrl($bucketName, $file);
38     echo "<li><a href=\"$fileUrl\">download>$file</a></li>";
39 }
40 echo "</ul>";
41 echo "<p>Numero totale di stipendi presenti: " .
42     count($pdfFiles) . "</p>";
43 ?>
44 </body>
45 </html>

```

2.5 Json

- Qual é l'output del seguente codice PHP?
`json = '{"matr": "123456", "voti": {"VDM": 28, "SB": 29}}'; print_r(json_decode($json));`
 l'output é un'array php
- Come si rappresenta in JSON il vettore PHP array('primo' → 'Farina', 'secondo' → 'Fangio') ?
- Come si codifica in JSON il seguente vettore PHP? Quale aspetto avrà dopo la codifica?

```

1 <?php
2
3 // 1.
4 stdClass Object
5 $json= '{"matr": "123456", "voti": {"VDM": 28, "SB": 29}}';
6 print_r(json_decode($json));
7
8 // 2.
9
10 "primo": "Farina",
11 "secondo": "Fangio"
12 }
13
14 // 3.
15 $contatto=array( cognome => Potter , nome=> Harry , eta =>12, domicilio=> Hogwarts )
16 $json_contatto = json_encode($contatto);
17 echo $json_contatto;
18 {
19     "cognome": "Potter",
20     "nome": "Harry",
21     "eta": 12,
22     "domicilio": "Hogwarts"
23 }
24
25 ?>

```

```

) php temp.php
stdClass Object
(
    [matr] => 123456
    [voti] => stdClass Object
        (
            [VDM] => 28
            [SB] => 29
        )
)

```

2.6 Funzioni

Si implementi in PHP una funzione log-accessi() con le seguenti caratteristiche, atte ad implementare un meccanismo di log per pagine web:

1. la funzione prende dal vettore \$_SERVER alcuni campi che descrivono la chiamata http: QUERY_STRING, REMOTE_ADDR, REQUEST_TIME, HTTP_USER_AGENT;
2. la funzione stabilisce una connessione con DynamoDB su AWS (per semplicità, il config.php é da considerarsi già impostato)
3. i dati ricevuti vengono caricati su una tabella pre-esistente denominata "ACCESSI", che ha per chiave primaria un campo denominato "chiave", costituito dalla concatenazione del contenuto del campo REMOTE_ADDR con un numero casuale (NB: la funzione rand() di PHP restituisce un numero casuale...).
4. La funzione deve restituire la chiave usata per l'inserimento.

```

1 <?php
2 require_once "config.php";
3 require 'vendor/autoload.php';
4
5 use Aws\DynamoDb\DynamoDbClient;
6 use Aws\Exception\AwsException;
7
8 $credentials = new Aws\Credentials\Credentials('key', 'secret');
9
10
11
12 function log_accessi() {
13     $QueryString = $_SERVER['QUERY_STRING'];
14     $RmAdr = $_SERVER['REMOTE_ADDR'];
15     $RqTime = $_SERVER['REQUEST_TIME'];
16     $HUser = $_SERVER['HTTP_USER_AGENT'];
17
18     // Creazione Client DynamoDb
19     $client = new Aws\DynamoDb\DynamoDbClient([
20         'version' => 'latest',
21         'region' => 'eu-south-1',
22         'credentials' => $credentials
23     ]);
24
25     $table = "rubrica";
26     $lista = $client -> listTables();
27     //print_r($lista)
28     foreach ($lista['TableNames'] as $t) {
29         echo $t . "\n";
30     }
31
32     // Creazione item
33
34     $homer = array(
35         'nome' => array('S' => 'Homer'), // tipo => dato
36         'cognome' => array('S' => 'Simpson'),
37         'indirizzo' => array('S' => '724□Evergreen□Terrace'),
38         'luogo' => array('S' => 'Springfield'),
39         'telefono' => array('N' => '123456789')
40     );
41
42
43
44
45
46 }
47
48
49
50 ?>

```

2.7 Logger

Si implementi in PHP uno script logger.php con le seguenti caratteristiche:

1. lo script riceve in input in modalità POST una serie di campi relativi agli accessi ad un sito, che includono: IP , TIME , AGENT , RESOURCE ;
2. se i campi non sono tutti presenti o sono vuoti, l'esecuzione termina e mostra una segnalazione di errore;
3. lo script stabilisce una connessione con DynamoDB su AWS (per semplicità, il config.php da considerarsi già impostato);
4. i dati ricevuti vengono caricati su una tabella pre-esistente denominata " LOGGER ", che ha per chiave primaria un campo denominato " KEY ", costituito dalla concatenazione del contenuto del campo IP con il campo TIME;
5. al termine del caricamento si deve stampare la chiave usata per l'inserimento.

```

1 <?php
2
3 require 'vendor/autoload.php';
4 require 'config.php';
5 use Aws\DynamoDb\DynamoDbClient;
6 use Aws\DynamoDb\Exception\DynamoDbException;
7 if ($_SERVER['REQUEST_METHOD'] == 'POST') {
8     $ip = isset($_POST['IP']) ? $_POST['IP'] : null;
9     $time = isset($_POST['TIME']) ? $_POST['TIME'] : null;
10    $agent = isset($_POST['AGENT']) ? $_POST['AGENT'] : null;
11    $resource = isset($_POST['RESOURCE']) ? $_POST['RESOURCE'] : null;
12
13    if (!$ip || !$time || !$agent || !$resource) {
14        die('Errore: Tutti i campi devono essere presenti e non
15        vuoti.');
```

2.8 Matricola JSON

1. lo script riceve in input in modalità GET il parametro matricola
2. nel caso il parametro matricola sia assente o vuoto, l'esecuzione termina con un messaggio appropriato
3. altrimenti lo script stabilisce una connessione con DynamoDB su AWS
4. lo script estrae il record dalla tabella studenti(matricola, nome, cognome, data_nascita, corso) corrispondente al valore memorizzato nel parametro matricola
5. lo script emette in output una stringa JSON contenente le informazioni del record come segue: { "matricola": "...", "nome": "...", "cognome": "...", "data_nascita": "...", "corso": "..." }

Ad esempio, se viene trovata una corrispondenza per la matricola cercata: { "matricola": "125987", "nome": "Mario", "cognome": "Rossi", "data_nascita": "2000-11-12", "corso": "Lettere e Filosofia" } Altrimenti, ovvero, se non esiste nessuno studente con la matricola cercata, viene emesso quanto segue: { "matricola": "-", "nome": "-", "cognome": "-", "data_nascita": "-", "corso": "-" }

```

1 <?php
2
3 require_once("config.php");
4 require 'vendor/autoload.php';
5 use Aws\DynamoDb\DynamoDbClient;
6 use Aws\Exception\AwsException;
7 $config = [
8     'region' => 'eu-south-1',
9     'version' => 'latest',
10    'credentials' => [
11        'key' => 'your-access-key-id',
12        'secret' => 'your-secret-access-key',
13    ],
14];
15 $dynamodb = new DynamoDbClient($config);
16 if (!isset($_GET['matricola']) || empty(trim($_GET['matricola']))) {
17     die(json_encode([
18         "matricola" => "-",
19         "nome" => "-",
20         "cognome" => "-",
21         "data_nascita" => "-",
22         "corso" => "-"
23     ]));
24 }
25 $matricola = trim($_GET['matricola']);
26 try {
27     $result = $dynamodb->getItem([
28         'TableName' => 'studenti',
29         'Key' => [
30             'matricola' => ['S' => $matricola]
31         ]
32     ]);
33     if (isset($result['Item'])) { //Converti dati in array associativo
34         $item = $result['Item'];
35         $data = [
36             "matricola" => $item['matricola']['S'],
37             "nome" => $item['nome']['S'],
38             "cognome" => $item['cognome']['S'],
39             "data_nascita" => $item['data_nascita']['S'],
40             "corso" => $item['corso']['S']
41         ];
42     } else { //Nessun record trovato per la matricola
43         $data = [
44             "matricola" => "-",
45             "nome" => "-",
46             "cognome" => "-",
47             "data_nascita" => "-",
48             "corso" => "-"
49         ];
50     }
51     header('Content-Type: application/json'); //Emissione risultato
52     JSON
53     echo json_encode($data);
54 } catch (AwsException $e) {
55     die(json_encode([
56         "matricola" => "-",
57         "nome" => "-",
58         "cognome" => "-",
59         "data_nascita" => "-",
60         "corso" => "-"
61     ]));
62 }
63 ?

```

2.9 php JPEG

Scrivere il minimo necessario affinché uno script PHP generi una risorsa che per il browser che la riceve sia in formato JPEG.

```

1 <?php
2
3 // Imposta l'intestazione del contenuto per JPEG
4 header('Content-Type: image/jpeg');
5 // Crea un'immagine di esempio
6 $width = 200;
7 $height = 100;
8 $image = imagecreatetruecolor($width, $height);
9 // Imposta un colore di sfondo
10 $backgroundColor = imagecolorallocate($image, 255, 255, 255); // Bianco
11 imagefill($image, 0, 0, $backgroundColor);
12 // Invia l'immagine al browser
13 imagejpeg($image);
14 // Libera la memoria
15 imagedestroy($image);
16 ?>

```

```

1 <html>
2 <head>
3 <title>Rubrica DynamoDB</title>
4 </head>

```

```

5  <body>
6  <?php
7  require 'vendor/autoload.php';
8  require 'config.php';
9  use Aws\DynamoDb\DynamoDbClient;
10 use Aws\Exception\AwsException;
11
12 $credentials = new Aws\Credentials\Credentials('chiave', 'segreto');
13
14 //Crea un client per DynamoDb
15
16 $client = new Aws\DynamoDb\DynamoDbClient([
17     'version' => 'latest',
18     'region' => 'eu-south-1',
19     'credentials' => $credentials
20 ]);
21
22 $table="rubrica"; //twc24
23 $lista = $client->listTables();
24
25 //print_r($lista);
26
27 foreach ($lista['TableNames'] as $t) {
28     echo $t . "\n";
29 }
30
31 // Creazione item
32
33 $homer=array(
34     'nome' => array('S'=>'Homer'), // tipo => dato
35     'cognome' => array('S'=>'Simpson'),
36     'indirizzo' => array('S'=>'742_Evergreen_Terrace'),
37     'luogo' => array('S'=>'Springfield'),
38     'telefono' => array('N'=>'123456789')
39 );
40
41 $rs = $client -> putItem(array(
42     'TableName' => $table,
43     'Item' => $homer));
44
45 var_dump($rs);
46
47 // Inserisci item nella tabella
48 $rs = $client->putItem(array(
49     'TableName' => $table,
50     'Item' => $marge)
51 );
52
53 // creo iterator per nome == Homer
54 $iterator = $client->getIterator('Query', array(
55     'TableName' => $table,
56     'KeyConditions' => array(
57         'nome' => array(
58             'AttributeValueList' => array(array('S' =>'Homer')),
59             'ComparisonOperator' => 'EQ' )
60         )
61     ));
62
63 // ComparisonOperator: EQ|NE|IN|LE|LT|GE|GT|BETWEEN|NOT_NULL|NULL|CONTAINS| NOT_CONTAINS|BEGINS_WITH, // REQUIRED ],
64
65 foreach ($iterator as $item) {
66     echo $item['nome']['S'] . "\n";
67     echo $item['cognome']['S'] . "\n";
68     echo $item['indirizzo']['SS'] . "\n";
69 }
70 // Creare tabella
71 try {
72     $client->createTable(array(
73         'TableName' => "nuovatabella",
74         'AttributeDefinitions' => array(
75             array( 'AttributeName' => 'nome', 'AttributeType'
76                 => 'S' ),
77             array( 'AttributeName' => 'cognome',
78                 'AttributeType' => 'S' )
79         ),
80         'KeySchema' => array(
81             array( 'AttributeName' => 'nome', 'KeyType' =>
82                 'HASH' ),
83             array( 'AttributeName' => 'cognome', 'KeyType' =>
84                 'RANGE' )
85         ),
86         'ProvisionedThroughput' => array( 'ReadCapacityUnits'
87             => 5, 'WriteCapacityUnits' => 5 )
88     ));
89     // NB si pu modificare a posteriori
90 } catch (Aws\DynamoDb\Exception\DynamoDbException $e) {
91     echo "Non posso creare la tabella: <br/>";
92     echo $e->getMessage() . "<br/>";
93 }
94 // Estrarre dati da tabella
95 $lista = $client->scan(array('TableName'=>$table));
96 foreach($lista['Items'] as $i)
97     print_r($i);
98 'ScanFilter' => array(
99     'cognome' => array(
100         'AttributeValueList' => array(array('S' =>

```

```

101     'Simpson')),
102     'ComparisonOperator' => 'CONTAINS' ),
103     'nome' => array(
104         'AttributeValueList' => array(array('S' => 'Bart')),
105         'ComparisonOperator' => 'EQ' )
106 );
107 $filtro = array(
108     'cognome' => array(
109         'AttributeValueList' => array(array('S' => 'Simpson' )
110     ),
111     'ComparisonOperator' => 'CONTAINS' ),
112     // 'nome' => array(
113     // 'AttributeValueList' => array(
114     // array('S' => 'Homer')),
115     // 'ComparisonOperator' => 'EQ' )
116 );
117 $lista = $client->scan(array('TableName'=>$table,
118     'ScanFilter'=>$filtro));
119 foreach($lista['Items'] as $i)
120     print_r($i);
121 ?>
122 </body>
123 </html>

```

2.10 UpFile

1. Lo script deve consentire l'upload di un file di testo tramite form. Il file é composto da linee con il seguente formato:
isbn#titolo#autore#data di pubblicazione#editore#numero di pagine Ad esempio:
 - 8807900769#Moby Dick#Herman Melville#1994-10-12#Adelphi#588
 - 8878994227#Frankenstein#Mary Shelley#2010-10-15#Barbera#176
2. lo script legge il contenuto del file, ed emette un messaggio di errore "File non conforme" se il formato di cui sopra non é rispettato (cioé il numero di colonne non é corretto).
3. Lo script visualizza una tabella HTML con le informazioni lette dal file (una riga per libro ed una colonna per attributo, ovvero, ISBN, titolo, autore, data di pubblicazione, editore e numero di pagine). In fondo alla pagina sono presenti due bottoni, etichettati "Importazione" ed "Annulla".
4. Se viene cliccato il bottone "Importazione", lo script stabilisce una connessione con DynamoDB su AWS (per semplicitá, il config.php é da considerarsi già impostato), ed i dati del file vengono caricati su una tabella pre-esistente denominata "libreria", che ha per chiave primaria un campo denominato "ISBN".
5. Al termine dell'importazione si deve far comparire un messaggio che dice quanti item sono presenti in tutto nella tabella

```

1 <html>
2 <head>
3     <title> UpFile </title>
4 </head>
5
6 <body>
7
8 <h2> Carica il file dei libri </h2>
9
10 <form method="post" action="upfile.php" enctype="multipart/form-data">
11
12     Seleziona il file
13     <br>
14     <input type="file" name="FileLibro" accept=".txt">
15     <br>
16     <input type="submit" value="upload">
17
18 </form>
19
20 <?php
21
22 function format($lines) {
23     foreach ($lines as $line) {
24         $columns = explode('#', $line);
25         if(count($columns) != 6) {
26             return false;
27         }
28     }
29     return true;
30 }
31
32 function Table($lines) {
33     echo '<h1>ElencoLibri</h1>';
34     echo '<table border="1">';
35     echo '<tr><th>ISBN</th>
36     <th>Titolo</th>
37     <th>Autore</th>
38     <th>DataDiPubblicazione</th>
39     <th>Editore</th>

```

```

40 <th>Numero di Pagine</th></tr>';
41 foreach($lines as $line) {
42     $columns = explode('#', $line);
43     echo '<tr>';
44     foreach ($columns as $column) {
45         echo '<td>' . htmlspecialchars($column) . '</td>';
46     }
47     echo '</tr>';
48 }
49 echo '</table>';
50
51 echo '<form action="import.php" method="post">';
52 echo '<input type="hidden" name="lines" value="' . htmlspecialchars(implode("\n", $lines)) . '>';
53 echo '<input type="submit" name="action" value="Importazione">';
54 echo '<input type="submit" name="action" value="Annulla">';
55 echo '</form>';
56 }
57
58 if($_SERVER['REQUEST_METHOD'] == 'POST') {
59     if(isset($_FILES['FileLibro']) && $_FILES['FileLibro']['error'] == 0) {
60         $fileTempName = $_FILES['bookfile']['tmp_name'];
61         $fileContent = file_get_contents($fileTempName);
62         $lines = explode("\n", $fileContent);
63         $lines = array_filter($lines, 'trim'); //rimozione righe vuote
64
65         if(!format($lines)) {
66             die("File non conforme");
67         } else {
68             renderTable($lines);
69         }
70     } else {
71         die("Errore durante il caricamento del file.");
72     }
73 }
74 ?>
75
76 <?php
77 //Dynamo
78 require_once 'config.php';
79 require 'vendor/autoload.php';
80
81 use Aws\DynamoDb\DynamoDbClient;
82 use Aws\Exception\AwsException;
83
84 if($_SERVER['REQUEST_METHOD'] == 'POST') {
85     if($_POST['action'] == 'Annulla') {
86         echo "Importazione annullata.";
87         exit;
88     }
89     $lines = explode("\n", $_POST['lines']);
90     $lines = array_filter($lines, 'trim');
91
92     //Config del client DynamoDB
93
94     $sdk = new Aws\Sdk([
95         'region' => AWS_REGION,
96         'version' => 'latest',
97         'credentials' => [
98             'key' => AWS_ACCESS_KEY_ID,
99             'secret' => AWS_SECRET_ACCESS_KEY,
100         ]
101     ]);
102
103     $dynamoDbClient = $sdk -> createDynamoDb();
104
105     //import dei dati
106
107     $tableName = 'Libreria';
108     foreach ($lines as $line) {
109         $columns = explode('#', $line);
110         try {
111             $dynamoDbClient -> putItem([
112                 'TableName' => $tableName,
113                 'Item' => [
114                     'ISBN' => ['S' => $columns[0]],
115                     'Titolo' => ['S' => $columns[1]],
116                     'Autore' => ['S' => $columns[2]],
117                     'Data di Pubblicazione' => ['S' => $columns[3]],
118                     'Editore' => ['S' => $columns[4]],
119                     'NumeroPagine' => ['N' => $columns[5]],
120                 ]
121             ]);
122         } catch (AwsException $e) {
123             echo "Errore durante l'importazione dei dati: " . $e -> getMessage();
124             exit;
125         }
126     }
127
128     //Conta il numero di item nella tabella
129     try {
130         $result = $dynamoDbClient -> scan(['TableName' => $tableName]);
131         $itemCount = ($result['Items']);
132         echo "Importazione completata. Numero totale di item presenti nella tabella: " . $itemCount;
133     } catch (AwsException $e) {
134         echo "Errore durante il conteggio degli item: " . $e -> getMessage();
135     }

```

```

136     }
137 }
138 ?>
139 </body>
140 </html>

```

2.11 Web Services

1. creare un requestor per un web service descritto dal file WSDL raggiungibile all'URI <http://www.uniud.it/servizio.wsdl>
2. Creare un web service provider descritto da un file WSDL raggiungibile all'URI <http://ws.it/log.wsdl>, e che espone le funzioni `log($pagina, $ip, $quando)` e `datiAccesso($id)` (ove `salva` e `recupera` sono già definite in un file `funzioni.php`; `log()` restituisce un ID, `datiAccesso()` restituisce un array con pagina, IP e data)

```

1 <?php
2
3 $wsdl = "http://www.uniud.it/servizio.wsdl";
4 try {
5     $client = new SoapClient($wsdl); // Crea un client SOAP usando il
6     file WSDL
7     $params = array('param1'=>'value1', 'param2'=>'value2'); //
8     Parametri chiamata esempio
9     $response = $client->getInfo($params); // Sostituire 'getInfo' con
10    il metodo effettivo
11    print_r($response); // Stampa risposta
12 } catch (SoapFault $fault) {
13     echo "Errore SOAP: " . $fault->getMessage();
14 }
15 ?>
16
17 <?php
18
19 require 'funzioni.php';
20 class LogService
21 {
22     public function log($pagina, $ip, $quando) {
23         return salvaLog($pagina, $ip, $quando);
24     }
25     public function datiAccesso($id) {
26         return recuperaLog($id);
27     }
28 }
29 // Creazione del server SOAP
30 $options = array('uri' => "http://ws.it/log.wsdl");
31 $server = new SoapServer("http://ws.it/log.wsdl", $options);
32 $server->setClass("LogService");
33 $server->handle();
34 ?>

```

2.12 Shortcut per clients

```

1 <?php
2
3 // Inizializzazione della connessione DynamoDb con Aws
4
5 require 'vendor/autoload.php';
6 require_once 'config.php';
7
8 use Aws\DynamoDb\DynamoDbClient;
9 use Aws\Exception\AwsException;
10
11 $credentials = new Aws\Credentials\Credentials (
12     'chiave',
13     'segreto'
14 );
15
16 // Creazione Client per DynamoDb
17
18 $client = new Aws\DynamoDb\DynamoDbClient ([
19     'version' => 'latest',
20     'region' => 'eu-south-1',
21     'credentials' => $credentials
22 ]);
23
24 try {
25     //code.. dipendentemente dall'esercizio
26 } catch (Aws\DynamoDb\Exception\DynamoDbException $e) {
27     echo "Errore, impossibile... esercizio...";
28     echo $e -> getMessage() . "<br/>";
29 }
30 ?>
31
32 <?php
33
34 // Inizializzazione della connessione S3 con Aws
35

```

```

36 // import uguali tranne:
37
38 use Aws\S3\S3Client;
39
40 $credentials = new Aws\Credentials\Credentials (
41     'chiave',
42     'segreto'
43 );
44
45 $s3= new Aws\S3\S3Client([
46     'version' => 'latest',
47     'region' => 'eu-south-1',
48     'credentials' => $credentials
49 ]);
50
51 $bucketName = 'fornito_da_consegna';
52 $filename = 'nomefile_da_caricare'; //struttura fornita da consegna
53
54 try {
55     //code.. dipendentemente dall'esercizio
56     $result = $s3->putObject([ // Esempio di utilizzo di funzione
57         'Bucket' => '$bucketName',
58         'Key' => '$filename',
59         'SourceFile' => '$tempfilename' // = $file['tmp_name'];
60     ], 'ACL' => 'public-read' // opzionale per rendere il file pubblico
61 );
62
63 } catch (AwsException $e) {
64     echo "Errore, impossibile...esercizio...";
65     echo $e -> getMessage() . "<br/>";
66 }
67 ?>
68
69 <?php
70
71 // Inizializzazione SOAP con Aws
72
73 $wsdl = "https://Uri_del_WSDL_da_richiamare";
74
75 try{
76     // init client soap
77     $requestor = new SoapClient($wsdl);
78
79     // Esempio di applicazione su funzione:
80     $cambio = $requestor -> GetRate(
81         array(
82             "CurrencyFrom" => "EUR",
83             "CurrencyTo" => "USD"
84         )
85     );
86     echo "Client SOAP inizializzato con successo.\n";
87     print("Cambio Euro-Dollaro: " . $cambio->GetRateResponse . "\n");
88 } catch (SoapFault $e) {
89     echo "Errore nell'inizializzazione del client SOAP: " . $e->getMessage() . "\n";
90 }
91 ?>

```

3 Domande di teoria

Di seguito sono riportate le risposte alle domande di teoria basate esclusivamente sulle informazioni contenute nelle tracce d'esame fornite.

Domande su Cloud

- Costo di un server in Cloud rispetto all'hosting tradizionale: La domanda posta è se "Un server in cloud costa sempre meno di un server in hosting presso un provider tradizionale". Le fonti presentano questa come un'affermazione da valutare, ma non forniscono una risposta esplicita o una giustificazione che confermi o neghi questa dichiarazione.
- Uso di AWS S3 per il backup privato: L'affermazione "Non posso usare AWS S3 per memorizzare il backup privato dei file del mio computer" è contraddetta dalle fonti. Le fonti affermano esplicitamente che è possibile "usare AWS S3 per memorizzare il backup privato dei file del mio computer". Pertanto, l'affermazione originale è falsa.
- Uso di AWS Lambda per implementare un sito web: La domanda posta è se sia possibile "usare AWS Lambda per implementare un sito web". Le fonti presentano questa come un'affermazione da valutare, ma non contengono informazioni che confermino o neghino esplicitamente questa possibilità.
- Uso di AWS S3 per ospitare pagine PHP: Le fonti pongono la domanda se sia possibile "usare AWS S3 per ospitare pagine PHP". Questa è presentata come un'affermazione da valutare, ma le fonti non forniscono una risposta esplicita o una giustificazione a riguardo.
- Installazione di sistemi operativi su EC2: La domanda posta è se "Su una risorsa EC2 posso installare qualsiasi sistema operativo". Le fonti presentano questa come un'affermazione da valutare, ma non contengono informazioni che confermino o neghino esplicitamente questa possibilità.

- Cambio di regione di una macchina AWS EC2: Una domanda posta é se sia possibile "cambiare la regione di una macchina AWS EC2 senza riavviare la macchina" . Le fonti presentano questa come un'affermazione da valutare, ma non forniscono una risposta esplicita o una giustificazione a riguardo.
- Costo delle risorse EC2 On-Demand vs Spot: Riguardo all'affermazione "Una risorsa on-demand EC2 costa sempre meno di una risorsa spot" , le fonti presentano un'affermazione contraddittoria. Un'altra sezione delle fonti afferma che "Una risorsa spot EC2 costa sempre meno di una risorsa on-demand" . Questo implica che la dichiarazione originale é falsa.
- Licenza del sistema operativo su server Cloud: La domanda "Se ho un server su Cloud, non pago la licenza per il sistema operativo" é presentata come un'affermazione da valutare. Le fonti non contengono informazioni che confermino o neghino esplicitamente questa affermazione.
- Availability Zone e Regioni: La domanda posta é se "Una Availability Zone può essere composta da più regioni" . Le fonti presentano questa come un'affermazione da valutare, ma non forniscono una risposta esplicita o una giustificazione a riguardo.
- Heroku e IAAS: Riguardo all'affermazione "Heroku fornisce un servizio IAAS" , le fonti non contengono alcuna informazione su Heroku o sui servizi IAAS (Infrastructure as a Service).
- Riavvio della macchina EC2 cambiando Availability Zone: La domanda posta é se "Se cambio la availability zone di una macchina AWS EC2 devo riavviare la macchina" . Le fonti presentano questa come un'affermazione da valutare, ma non forniscono una risposta esplicita o una giustificazione a riguardo.
- Database NoSQL e Tabelle: Riguardo all'affermazione "In un database NoSQL non esistono le tabelle" , le fonti non contengono informazioni che confermino o neghino esplicitamente questa affermazione.
- AWS Relational Database Service (RDS) come unico servizio relazionale: La domanda "AWS Relational Database Service (RDS) é l'unico servizio con cui posso implementare un database relazionale su AWS" é presentata come un'affermazione da valutare. Le fonti non contengono informazioni che confermino o neghino esplicitamente questa affermazione.

Domande su PHP e JSON

- Questo tipo di rappresentazione é un oggetto JSON, dove le chiavi sono stringhe e i valori corrispondono ai tipi di dato PHP.
- Tipizzazione dinamica in PHP: La domanda posta é se "PHP consente di attribuire ad una stessa variabile valori di tipi di dato diversi all'interno di uno stesso script (ad esempio é possibile assegnare alla variabile \$a prima la stringa "abc" e poi l'intero 9)" . Le fonti presentano questa come un'affermazione da valutare, ma non contengono informazioni che confermino o neghino esplicitamente questa caratteristica di PHP.
- Invio di file tramite form con GET o POST: La dichiarazione "per l'invio di un file tramite form é indifferente utilizzare il metodo GET o POST" é implicitamente contraddetta dalle istruzioni presenti nelle fonti. Gli script per l'upload di file specificano sempre la ricezione in modalità POST , suggerendo che il metodo non sia indifferente per questa operazione.
- Salvataggio dati di sessione sul client: La domanda posta é se "le sessioni consentono di salvare i dati sul client" . La fonte presenta questa come un'affermazione da valutare. Le fonti non approfondiscono il meccanismo di funzionamento delle sessioni per confermare o smentire esplicitamente questa specifica affermazione.

Domande su Architettura e Framework

- Componenti principali di un'architettura orientata ai servizi: La domanda "Quali sono i principali componenti di una architettura orientata ai servizi" é presente nelle fonti, ma non viene fornita alcuna informazione o risposta a riguardo.
- Laravel per applicazioni lato client: Riguardo all'affermazione "Un framework come Laravel mi permette di realizzare un'applicazione completamente lato client" , le fonti non contengono informazioni su Laravel o sulla sua capacità di sviluppare applicazioni lato client.